



UNIVERSIDAD DE LOS LAGOS

LABORATORIO PRÁCTICO N°2

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
TALLER DE INGENIERÍA INFORMÁTICA
CAMPUS MEYER, CHILE

Gabriel Lebien Lebien
Simón Pérez Cisternas
Sebastián Valderas Mancilla

4 de junio de 2026



5 años **UNIVERSIDAD ACREDITADA**
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Índice general

1. Introducción	1
2. Desarrollo	2
2.1. Evidencia de Desarrollo	2
2.1.1. Construcción de la Imagen (docker build)	2
2.1.2. Levantamiento de Contenedores (docker-compose up)	4
2.1.3. Revisión de Contenedores (docker ps)	4
2.1.4. Aplicación web	5
3. Conclusión	7
Configuración Docker	8
Repositorio y Enlaces	10

Capítulo 1

Introducción

En este informe, documentaremos el desarrollo del Laboratorio Práctico N°2. Este laboratorio consiste en crear una aplicación web dinámica que pueda hacer cada operación CRUD, dicha aplicación será implementada en un contenedor Docker. La aplicación web que será implementada consiste en un registro de productos, donde se pueden crear productos, borrar productos, editar productos.

Capítulo 2

Desarrollo

2.1. Evidencia de Desarrollo

A continuación, se presentan las capturas de pantalla de la consola Linux ejecutando los comandos fundamentales para el despliegue de la aplicación mediante Docker.

2.1.1. Construcción de la Imagen (docker build)

En esta captura se evidencia la ejecución del comando `docker build`. Este comando es el responsable de leer secuencialmente las instrucciones definidas en el archivo `Dockerfile` para compilar y generar una nueva imagen de contenedor. Durante la traza de la consola se puede observar cómo Docker descarga las capas de la imagen base, ejecuta las directivas internas (como la instalación de dependencias y la copia del código fuente) y, finalmente, empaqueta la aplicación en una imagen inmutable, etiquetada y lista para ser instanciada.

```

root@192.168.1.160:22 - Bitvise shell
=> sha256:f6e6fb3632696203fc5956e86e27974f859e091fad8d64099427e088d029e85f 2498 / 2498 0.4s
=> sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cd5577484ad75e68dc38e8acc1 328 / 328 0.4s
=> sha256:7996c9d2949eaf0ba880fc8fe74c069e95caae1f670ff2b86e9668921c611ca 8928 / 8928 0.5s
=> sha256:9a2c3b28bcf76a00fef150b921dec858040778b7e7d563664765286d0408608e 2558 / 2558 0.5s
=> sha256:1b4118522fd52eb7cc95ebae8f530b3a1b98113192ebc61ee973cd05b961e1ff 2.46kB / 2.46kB 0.3s
=> sha256:e694c55ba5aba821149fc8bc67d1f1ae38892d2150bac0980a069ceda8b29188 11.46MB / 11.46MB 213.1s
=> sha256:72950e9a0ba87c0c7e2c0895309ac3e71ee979339e2798eb43a5ca1632baef8 12.33MB / 12.33MB 251.3s
=> sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2430877af9840c5211 4888 / 4888 1.3s
=> sha256:bc3476dda7999ef33fbfbbef621ed871892ed96223ef0a8b4649f54f0fab69b 4888 / 4888 1.2s
=> sha256:d067a15f7ac10dbce34216e5c0ca2b275ede97675881157c6e42b8d0c87805e5 4358 / 4358 0.2s
=> sha256:f82bf7789d7ed61ec9e65d8692b087b0f8af7eabbc524375c958ca663132a46 4.23MB / 4.23MB 126.9s
=> sha256:cd80f85f90e767b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 2268 / 2268 0.2s
=> sha256:487e971560f085dc9521874e757ab1114f0ea303f54e70f2a5bca1ae53fc39 117.84MB / 117.84MB 2271.2s
=> sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328ea86026a7e781f57 2258 / 2258 1.5s
=> sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccae970 29.36MB / 29.78MB 2168.0s
=> extracting sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccae970 2.1s
=> extracting sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328ea86026a7e781f57 0.0s
=> extracting sha256:487e971560f085dc9521874e757ab1114f0ea303f54e70f2a5bca1ae53fc39 6.8s
=> extracting sha256:cd80f85f90e767b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 0.0s
=> extracting sha256:f82bf7789d7ed61ec9e65d8692b087b0f8af7eabbc524375c958ca663132a46 0.3s
=> extracting sha256:d067a15f7ac10dbce34216e5c0ca2b275ede97675881157c6e42b8d0c87805e5 0.0s
=> extracting sha256:bc3476dda7999ef33fbfbbef621ed871892ed96223ef0a8b4649f54f0fab69b 0.0s
=> extracting sha256:72950e9a0ba87c0c7e2c0895309ac3e71ee979339e2798eb43a5ca1632baef8 0.2s
=> extracting sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2430877af9840c5211 0.0s
=> extracting sha256:e694c55ba5aba821149fc8bc67d1f1ae38892d2150bac0980a069ceda8b29188 0.9s
=> extracting sha256:1b4118522fd52eb7cc95ebae8f530b3a1b98113192ebc61ee973cd05b961e1ff 0.0s
=> extracting sha256:9a2c3b28bcf76a00fef150b921dec858040778b7e7d563664765286d0408608e 0.0s
=> extracting sha256:f6e6fb3632696203fc5956e86e27974f859e091fad8d64099427e088d029e85f 0.0s
=> extracting sha256:7996c9d2949eaf0ba880fc8fe74c069e95caae1f670ff2b86e9668921c611ca 0.0s
=> extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cd5577484ad75e68dc38e8acc1 15.5s
[2/2] RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli 0.5s
=> exporting to image 0.5s
=> exporting layers 0.2s
=> exporting manifest sha256:ac7503bcb474d3529d24cddb743364e5fa56af3482552d6019dbd3ccbedb611 0.0s
=> exporting config sha256:f6aaf18b944a5559d4cb8de3eb58e66e5d55b8cf34ad4ac623495b4f9ba34d 0.0s
=> exporting attestation manifest sha256:04d2667587538ae4a276d09bfed2e7615c1d0d4010c4de02c2468735c553169 0.1s
=> exporting manifest list sha256:689bab7f74c934043291d91945aa9ccf7c556888992c58daeb5727c90e8cf2d 0.0s
=> naming to docker.io/library/taller-docker-web:latest 0.0s
=> unpacking to docker.io/library/taller-docker-web:latest 0.1s
=> resolving provenance for metadata file 15.5s
[+] build 1/1 2304.1s
w Image taller-docker-web Built
root@debian:~/taller-docker#

```

Figura 2.1: Ejecución del comando docker build.

```

root@192.168.1.160:22 - Bitvise shell
root@debian:~/taller-docker# nano app/Dockerfile
root@debian:~/taller-docker# docker compose build
WARN[0000] /root/taller-docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential
confusion
[+] Building 2304.0s (8/8) FINISHED
=> [internal] load local bake definitions 0.0s
=> reading from stdin 4988 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> transferring Dockerfile: 124B 0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache 4.0s
=> [internal] load .dockerignore 0.1s
=> transferring context: 2B 0.0s
=> [1/2] FROM docker.io/library/php:8.2-apache:sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea040a3b94f925b6130858dd 2283.2s
=> resolve docker.io/library/php:8.2-apache:sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea040a3b94f925b6130858dd 0.1s
=> sha256:f6e6fb3632696203fc5956e86e27974f859e091fad8d64099427e088d029e85f 2498 / 2498 0.4s
=> sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cd5577484ad75e68dc38e8acc1 328 / 328 0.4s
=> sha256:7996c9d2949eaf0ba880fc8fe74c069e95caae1f670ff2b86e9668921c611ca 8928 / 8928 0.5s
=> sha256:9a2c3b28bcf76a00fef150b921dec858040778b7e7d563664765286d0408608e 2558 / 2558 0.5s
=> sha256:1b4118522fd52eb7cc95ebae8f530b3a1b98113192ebc61ee973cd05b961e1ff 2.46kB / 2.46kB 0.3s
=> sha256:e694c55ba5aba821149fc8bc67d1f1ae38892d2150bac0980a069ceda8b29188 11.46MB / 11.46MB 213.1s
=> sha256:72950e9a0ba87c0c7e2c0895309ac3e71ee979339e2798eb43a5ca1632baef8 12.33MB / 12.33MB 251.3s
=> sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2430877af9840c5211 4888 / 4888 1.3s
=> sha256:bc3476dda7999ef33fbfbbef621ed871892ed96223ef0a8b4649f54f0fab69b 4888 / 4888 1.2s
=> sha256:d067a15f7ac10dbce34216e5c0ca2b275ede97675881157c6e42b8d0c87805e5 4358 / 4358 0.2s
=> sha256:f82bf7789d7ed61ec9e65d8692b087b0f8af7eabbc524375c958ca663132a46 4.23MB / 4.23MB 126.9s
=> sha256:cd80f85f90e767b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 2268 / 2268 0.2s
=> sha256:487e971560f085dc9521874e757ab1114f0ea303f54e70f2a5bca1ae53fc39 117.84MB / 117.84MB 2271.2s
=> sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328ea86026a7e781f57 2258 / 2258 1.5s
=> sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccae970 29.36MB / 29.78MB 2168.0s
=> extracting sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccae970 2.1s
=> extracting sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328ea86026a7e781f57 0.0s
=> extracting sha256:487e971560f085dc9521874e757ab1114f0ea303f54e70f2a5bca1ae53fc39 6.8s
=> extracting sha256:cd80f85f90e767b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 0.0s
=> extracting sha256:f82bf7789d7ed61ec9e65d8692b087b0f8af7eabbc524375c958ca663132a46 0.3s
=> extracting sha256:d067a15f7ac10dbce34216e5c0ca2b275ede97675881157c6e42b8d0c87805e5 0.0s
=> extracting sha256:bc3476dda7999ef33fbfbbef621ed871892ed96223ef0a8b4649f54f0fab69b 0.0s
=> extracting sha256:72950e9a0ba87c0c7e2c0895309ac3e71ee979339e2798eb43a5ca1632baef8 0.2s
=> extracting sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2430877af9840c5211 0.0s
=> extracting sha256:e694c55ba5aba821149fc8bc67d1f1ae38892d2150bac0980a069ceda8b29188 0.9s
=> extracting sha256:1b4118522fd52eb7cc95ebae8f530b3a1b98113192ebc61ee973cd05b961e1ff 0.0s
=> extracting sha256:9a2c3b28bcf76a00fef150b921dec858040778b7e7d563664765286d0408608e 0.0s
=> extracting sha256:f6e6fb3632696203fc5956e86e27974f859e091fad8d64099427e088d029e85f 0.0s
=> extracting sha256:7996c9d2949eaf0ba880fc8fe74c069e95caae1f670ff2b86e9668921c611ca 0.0s

```

Figura 2.2: Ejecución del comando docker build.

2.1.2. Levantamiento de Contenedores (docker-compose up)

La imagen muestra la inicialización de los servicios mediante el comando `docker-compose up` (ejecutado con el parámetro `-d` para operar en modo *detached* o segundo plano). A diferencia de levantar un contenedor de forma aislada, este comando actúa como orquestador leyendo el archivo `docker-compose.yml`. En la salida por consola se aprecia cómo el motor de Docker Compose gestiona automáticamente el ciclo de vida del entorno: primero creando las redes internas y los volúmenes necesarios para la persistencia de datos, y luego iniciando los contenedores asociados a la aplicación.

```

root@192.168.1.160:22 - Bitvise sterm
-> sha256:bc3476dda7999ef33bfbbef621ed871892ed96223ef0a8b6449f54f6ab69b 480B / 480B 1.2s
-> sha256:d067a15f7ac10dbce34216e5c0ca2b275ede97675081157c6e42b8d0c87805e5 435B / 435B 0.2s
-> sha256:f82bf7789d7ed61ec9e5d8692b087b0f8af7eabbca524375c958ca63132a46 4.23MB / 4.23MB 126.9s
-> sha256:5cd80f85f90e7677b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 226B / 226B 0.2s
-> sha256:5cd80f85f90e7677b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 117.84MB / 117.84MB 2271.2s
-> sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328aaa86026a7e781f57 225B / 225B 1.5s
-> sha256:5b4d6f92fcd4e911b7753c954fac965d48c40fe1075758d284148ccace970 29.36MB / 29.78MB 2168.0s
-> extracting sha256:5b4d6f92fcd4e911b7753c954fac965d48c40fe1075758d284148ccace970 2.1s
-> extracting sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328aaa86026a7e781f57 0.0s
-> extracting sha256:487e971560f0056dc9521874e757ab1114f0ea303f54e78f2a5bca1a6e53fc39 6.8s
-> extracting sha256:5cd80f85f90e7677b1345e07c60dc5e1da24abc7847b2caf62356638e849f25b 0.0s
-> extracting sha256:f82bf7789d7ed61ec9e5d8692b087b0f8af7eabbca524375c958ca63132a46 0.3s
-> extracting sha256:d067a15f7ac10dbce34216e5c0ca2b275ede97675081157c6e42b8d0c87805e5 0.0s
-> extracting sha256:bc3476dda7999ef33bfbbef621ed871892ed96223ef0a8b6449f54f6ab69b 0.0s
-> extracting sha256:72950e9a0ba87c0c7e2c0895309ac3e71eefc79339e2798eb43a5ca1632baef8 0.2s
-> extracting sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2438877af9840c5211 0.0s
-> extracting sha256:e694c5b45b8e21149fc08c6701f1e38892d1150a09980a069cde8b29108 0.9s
-> extracting sha256:1b4118722f852a07cc05bba8f530b3a1b08113192ebc61ea973cd059861c1ff 0.0s
-> extracting sha256:9a2c3b28cf76a08fef150b921dec8580a8778b7c7d56366d765286d0408608e 0.0s
-> extracting sha256:ffe6fb3632696203fc5956e80e27974f859e091fad8d64099427e888d029e85f 0.0s
-> extracting sha256:7596c9d2949eaf0ba880fc8fe74c069e95caae1f670ff2b8e6a9668923c611ca 0.0s
-> extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cd5577484a6d75e68dc38e8acc1 0.0s
[2/2] RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli 15.5s
-> exporting to image 0.5s
-> exporting layers 0.2s
-> exporting manifest sha256:ac7583bcb474d3520424c4db7433645fa56af24825522d019dbd3c3cbe611 0.0s
-> exporting config sha256:f6aaf18b944a5559d4cb84c3eb58e66a5d525b8c6f34a44ac623495b4f9ba34d 0.0s
-> exporting attestation manifest sha256:84d2667587538ae4a276d609bfcfd2e7651cde0d4018c4e82c2468735c553169 0.0s
-> exporting manifest list sha256:689bab7f74c9340432919d145a5afccf7c556888982c58daab572fc90e8cfd2 0.0s
-> naming to docker.io/library/taller-docker-web:latest 0.0s
-> unpacking to docker.io/library/taller-docker-web:latest 0.1s
-> resolving provenance for metadata file 0.0s
[+] build 1/1 2304.1s
  Image taller-docker-web Built
root@debian:~/taller-docker# docker compose up -d
WARN[0000] /root/taller-docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 15/15
  Image mariadb:latest Pulled 1371.0s
  Network taller-docker_red_app Created 0.2s
  Volume taller-docker_db_data Created 0.0s
  Container app_db_mariadb Started 1.3s
  Container app_web_php Started 1.8s
root@debian:~/taller-docker#

```

Figura 2.3: Ejecución del comando `docker-compose up`.

2.1.3. Revisión de Contenedores (docker ps)

En esta captura se emplea el comando `docker ps` para auditar el estado de los contenedores que se encuentran actualmente en ejecución. La tabla generada en la terminal es una herramienta de monitorización vital, ya que expone métricas y configuraciones en tiempo real. Entre los datos más relevantes se destaca el ID único del contenedor, la imagen base utilizada, el estado actual o tiempo de actividad (*STATUS*) y, de especial importancia técnica, el mapeo de puertos (*PORTS*), el cual verifica que el tráfico de red esté correctamente enrutado hacia la aplicación.

```

root@192.168.1.160:22 - Bitvise shell
=> sha256:487e971560f0056dc9521874e757ab1114f0ea303f54e70f2a5bca1a6e53fc39 117.84MB / 117.84MB 2271.2s
=> sha256:34ab7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328aa86020a7e781f57 2250 / 2250 1.5s
=> sha256:5b4d6ff92f4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 29.36MB / 29.78MB 2168.0s
=> extracting sha256:5b4d6ff92f4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 2.1s
=> extracting sha256:34ab7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328aa86020a7e781f57 0.0s
=> extracting sha256:487e971560f0056dc9521874e757ab1114f0ea303f54e70f2a5bca1a6e53fc39 6.8s
=> extracting sha256:5cd80f85f90e7677b1345e07c60dc5e1da24abc7847b2caff62356638e849f25b 0.0s
=> extracting sha256:f82bf7789d7ed61ec9e65d8692b087b0f8af7eabbc5a24375c958ca663132a46 0.3s
=> extracting sha256:d867a15f7ac10dbce34216e5c0ca2b275ede97675081157c6e42b8d0c87805e5 0.0s
=> extracting sha256:bc3476edda7999ef33bfbb9f621ed871892ed96223ef0a88b4649f54f0fab69b 0.0s
=> extracting sha256:72950e9a0ba87c0c7e2c0895309ac3e71eefc79339e2798eb43a5ca1632baef8 0.2s
=> extracting sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2438877af9840c5211 0.0s
=> extracting sha256:d094c5ba5ab021149fc0bc074f1fec38892d2150bac9900a069ceda0b29188 0.9s
=> extracting sha256:1b4118522fd52ab7cc95ebae8f530b1a1b08113192ebc61ea073c0d5b961a1ff 0.0s
=> extracting sha256:9a2c3b28bcf76a00fef150b921dec858048778b7e7d56366d765286d0408608e 0.0s
=> extracting sha256:ffe6fb3632696203fc5956e80e27974f859e091fa8d64099427e088d029e85f 0.0s
=> extracting sha256:7596c9d2949eaf0ba880fc8fe74c069e95caae1f670ff2b8e6a9668923c611ca 0.0s
=> extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cd5577484a6d75e6dc38e8ac1 0.0s
[2/2] RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli 15.5s
=> exporting to image 0.5s
=> exporting layers 0.2s
=> exporting manifest sha256:ac7583bcb474d3520d24c4db7433645fa56af34825522d019db43ccbedb611 0.0s
=> exporting config sha256:f6aaf18b944a5559d4cb8de3eb58e66e5d525b8cf34a44ac623495b4f9ba34d 0.0s
=> exporting attestation manifest sha256:04d2667587538ae4a276d609bfed2e7051cde0d4010c4e82c2468735c553169 0.1s
=> exporting manifest list sha256:689bab7f74c9340432919d1945aafccf7c55688982c58daeb572fc90e8cf2d 0.0s
=> naming to docker.io/library/taller-docker-web:latest 0.0s
=> unpacking to docker.io/library/taller-docker-web:latest 0.1s
=> resolving provenance for metadata file 0.0s
[+] build 1/1 2304.1s
v Image taller-docker-web Built
root@debian:~/taller-docker# docker compose up -d
WARN[0000] /root/taller-docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 15/15
v Image mariadb:latest Pulled 1371.0s
v Network taller-docker_red_app Created 0.2s
v Volume taller-docker_db_data Created 0.0s
v Container app_db_mariadb Started 1.3s
v Container app_web_php Started 1.8s
root@debian:~/taller-docker# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
22fe872bb16   taller-docker-web   "docker-php-entrypoin..." 54 seconds ago   Up 52 seconds   0.0.0.0:8880->80/tcp, [::]:8880->80/tcp   app_web_php
49da861931a0   mariadb:latest     "docker-entrypoint.s..." 54 seconds ago   Up 53 seconds   3306/tcp                 app_db_mariadb

```

Figura 2.4: Ejecución del comando docker ps.

2.1.4. Aplicación web

Finalmente podemos apreciar la aplicación web, la cual está alojada en un contenedor docker. Esta aplicación web utiliza MariaDB para almacenar los productos que se crean en la aplicación web. El backend fue desarrollado con PHP.

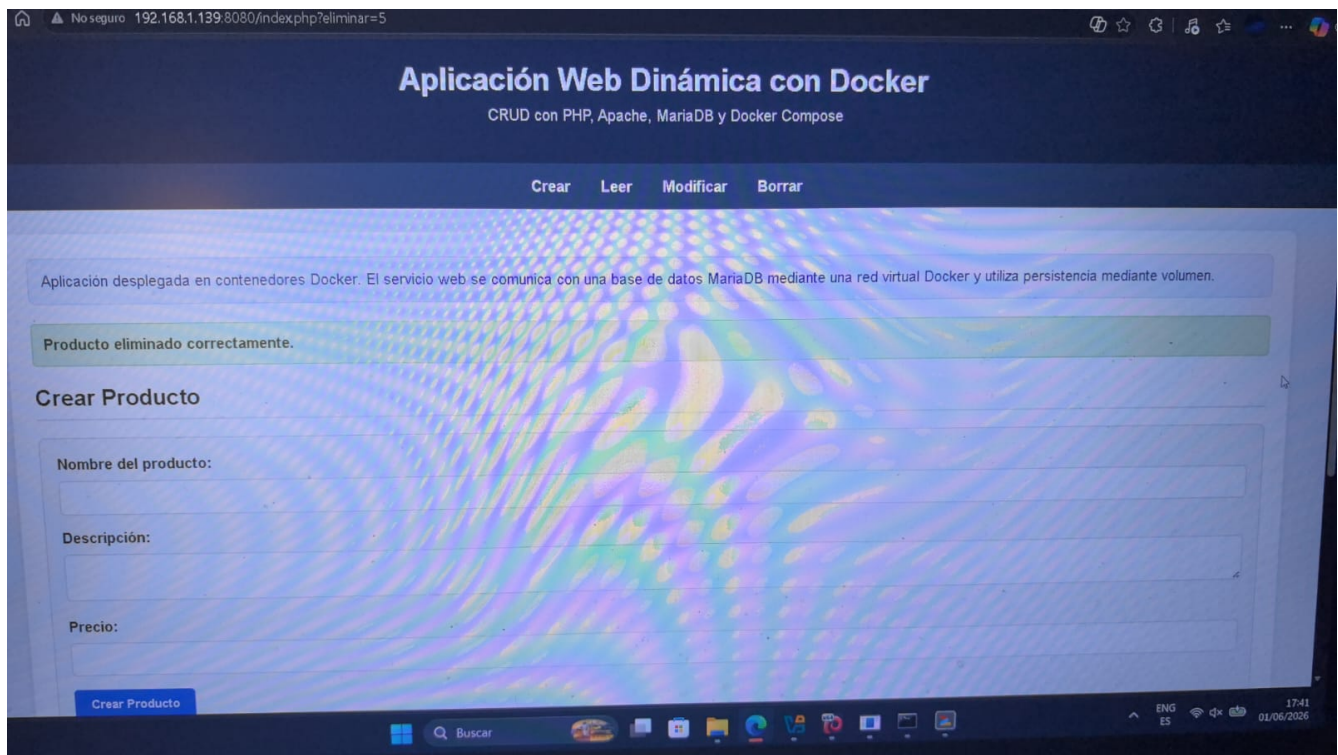


Figura 2.5: aplicacion web final.

Capítulo 3

Conclusión

En este laboratorio pusimos en práctica lo que habíamos estado trabajando en clases, con contenedores docker, máquinas virtuales. Pero esta vez trabajamos con aplicaciones web dinámicas y base de datos, agregando dificultad con respecto al trabajo que habíamos hecho en clases, donde principalmente se trabajó con aplicaciones web estáticas. En conclusión fue un laboratorio práctico bastante provechoso.

Configuración Docker

A continuación se presenta el código fuente de los archivos de configuración utilizados, los cuales siguen buenas prácticas como el uso de imágenes ligeras, persistencia de datos y variables de entorno.

Dockerfile

```
1 FROM php:8.2-apache
2
3 RUN docker-php-ext-install mysqli pdo pdo_mysql
4
5 EXPOSE 80
```

docker-compose.yml

```
1 services:
2   app:
3     build: ./app
4     container_name: app_web_php
5     ports:
6       - "8080:80"
7     volumes:
8       - ./app:/var/www/html
9     networks:
10      - red_app
11     depends_on:
12      - db
13     environment:
14       DB_HOST: db
15       DB_NAME: crud_db
16       DB_USER: usuario_web
```

```
17     DB_PASSWORD: password_seguro
18
19 db:
20     image: mariadb:11
21     container_name: app_db_mariadb
22     restart: always
23     environment:
24         MARIADB_ROOT_PASSWORD: root_password
25         MARIADB_DATABASE: crud_db
26         MARIADB_USER: usuario_web
27         MARIADB_PASSWORD: password_seguro
28     volumes:
29         - db_data:/var/lib/mysql
30         - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
31     networks:
32         - red_app
33
34 networks:
35     red_app:
36         driver: bridge
37
38 volumes:
39     db_data:
```

Repositorio y Enlaces

Repositorio GitHub

El código fuente de este proyecto, los archivos de configuración de Docker, el archivo `.gitignore` y el `README.md` se encuentran en el siguiente repositorio público:

- **Enlace al Repositorio:** <https://github.com/eseuvedev/Laboratorio-N-2-Taller-de-Ingenier-a->